Testes automatizados com Pytest

Para verificar o funcionamento das funcionalidades implementadas no sistema Virtual Língua, foram executados testes automatizados utilizando o framework Pytest, integrado ao banco de dados MySQL.

Os testes contemplaram operações de autenticação, controle de acesso, cadastro, edição e exclusão de registros, além de verificações de integridade dos dados, duplicidade de e-mails e rollback de transações.

Durante a execução, o banco de testes virtualingua_test foi recriado e populado automaticamente, garantindo um ambiente separado do banco principal do sistema.

Ao todo, foram executados 21 testes, todos aprovados, sem a identificação de falhas.

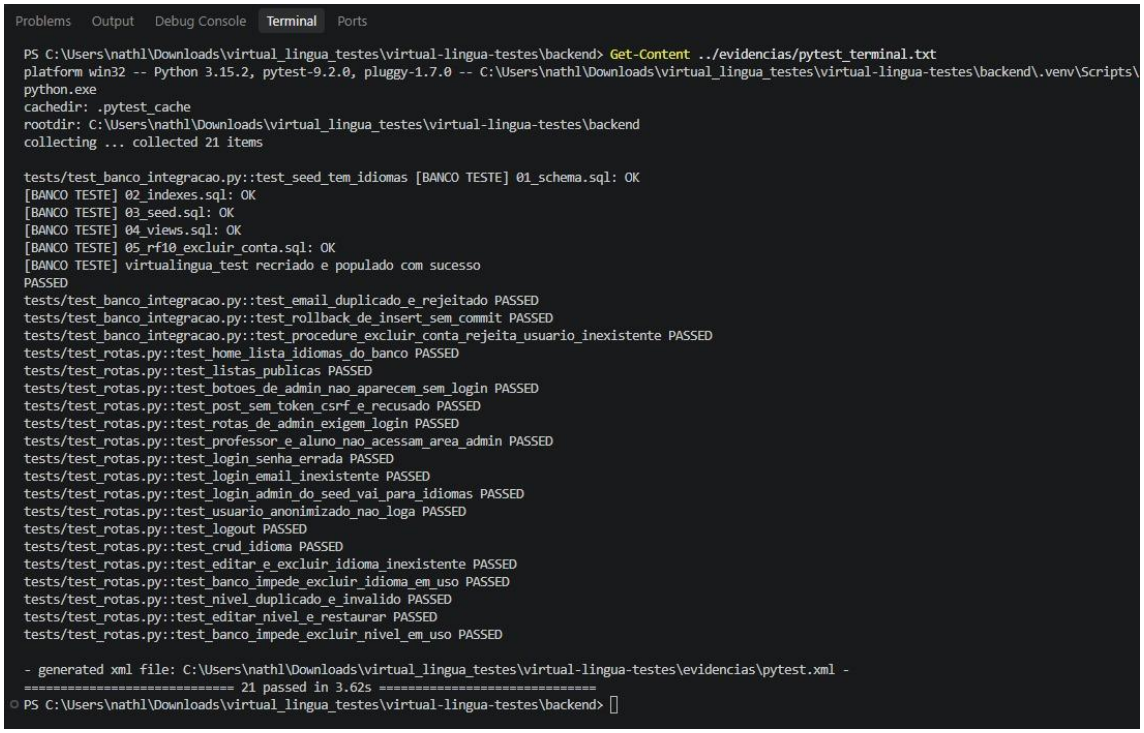


Figura 1 – Execução dos testes automatizados com Pytest. Foram executados 21 testes, todos aprovados, incluindo verificações de autenticação, operações CRUD, integridade do banco de dados, tratamento de e-mails duplicados e rollback de transações. O banco virtualingua_test foi recriado e populado com sucesso antes da execução.

Testes BDD com Behave

Para validar o comportamento do sistema a partir dos requisitos funcionais, foi utilizada a abordagem BDD (*Behavior-Driven Development*), com o framework Behave.

Foram implementados dois cenários relacionados ao cadastro de idiomas: o cadastro de um idioma válido e a tentativa de cadastro utilizando um código já existente. Esses cenários permitiram verificar tanto o funcionamento esperado quanto o tratamento de uma operação inválida.

Identificação e correção de falhas

Durante a execução inicial dos testes BDD, os dois cenários apresentaram falhas nas verificações realizadas pelos steps.

Após a análise, foi identificado um problema relacionado ao gerenciamento do token CSRF durante o processo de autenticação. O token utilizado inicialmente era invalidado após o login, ocasionando a rejeição das requisições de cadastro.

Para solucionar o problema, os steps foram ajustados para utilizar um token válido após a autenticação, preservando os mecanismos de segurança e sem necessidade de modificar o código principal desenvolvido pelos demais integrantes da equipe.

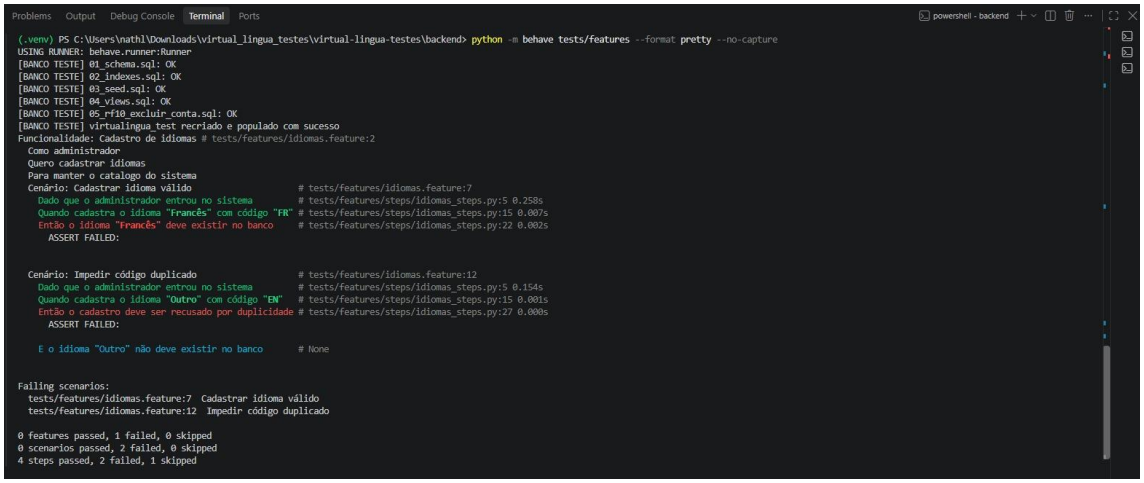


Figura 2 – Identificação de falhas na execução inicial dos testes BDD. Resultado da primeira execução com Behave, na qual os dois cenários apresentaram falhas de validação, posteriormente investigadas e corrigidas.

Validação após as correções

Após os ajustes, os cenários foram executados novamente, apresentando resultados positivos. A execução final registrou dois cenários aprovados, totalizando sete passos executados com sucesso e nenhuma falha.

O banco virtualingua_test também foi recriado e populado automaticamente antes da execução, permitindo realizar as verificações em um ambiente isolado.

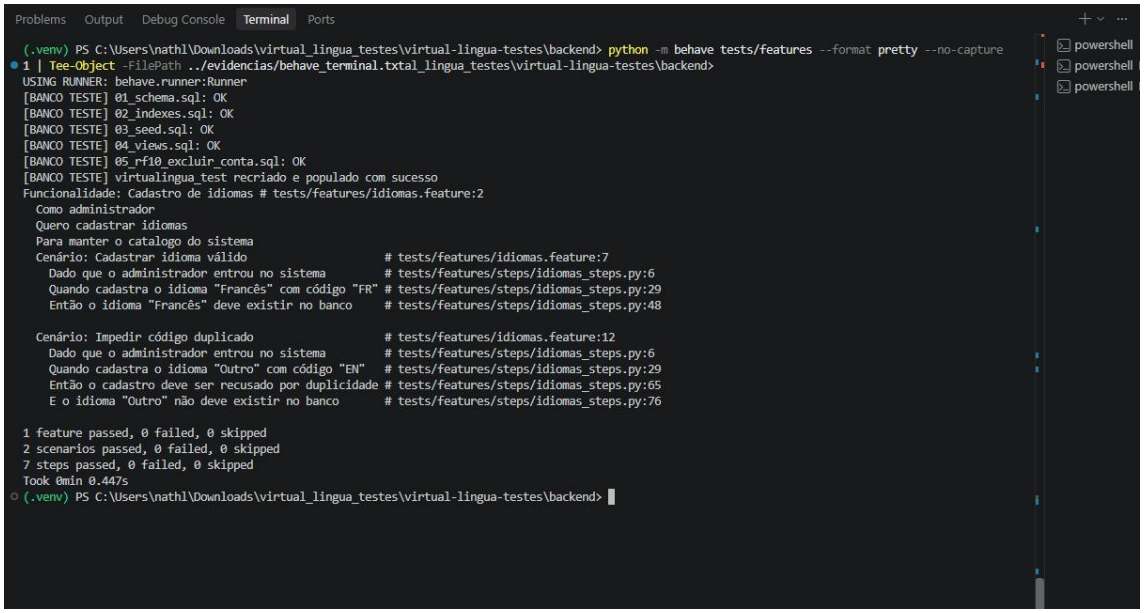


Figura 3 – Execução bem-sucedida dos testes BDD com Behave após as correções. Resultado da nova execução dos dois cenários de cadastro de idiomas, com aprovação dos sete passos e nenhuma falha, após a correção do gerenciamento do token CSRF.

Validação do banco de dados de testes

Além das execuções automatizadas, foi realizada uma consulta direta ao banco virtualingua_test por meio de Python, utilizando a conexão MySQL configurada no projeto.

A consulta confirmou a conexão com o banco de testes e identificou 2 idiomas, 19 usuários e 7 turmas cadastradas. Essa verificação complementou as evidências da execução automatizada, demonstrando que o banco separado estava acessível e continha os registros utilizados nos testes.

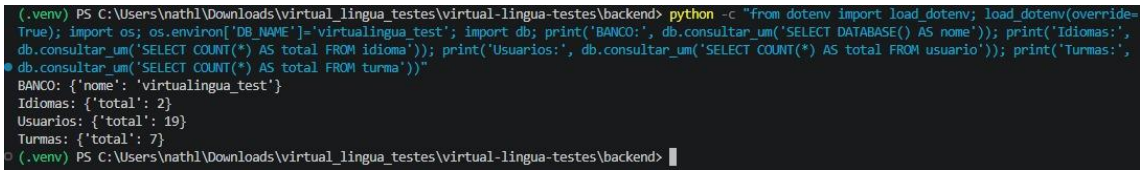


Figura 4 – Validação do banco de dados de testes. Consulta realizada via Python ao banco MySQL virtualingua_test, confirmando a conexão com o ambiente isolado e a presença de 2 idiomas, 19 usuários e 7 turmas cadastradas.

Considerações finais

Os resultados obtidos demonstraram o funcionamento dos testes automatizados implementados para as funcionalidades avaliadas do sistema Virtual Língua.

A utilização conjunta de Pytest e Behave permitiu verificar tanto operações e regras de integridade quanto cenários de comportamento descritos a partir dos requisitos funcionais. A identificação e posterior correção das falhas nos testes BDD também contribuíram para validar a configuração da automação, sem interferir nas funcionalidades desenvolvidas pelos demais integrantes.

A utilização de um banco de testes separado proporcionou maior controle sobre as execuções e reduziu o risco de alterações indevidas no ambiente principal.